

Conceitos de Integração e Entrega Contínua de Software

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br

Sumário

- DevOps;
- CI;
- Continuous Delivery;
- Continuous Deployment;
- Versionamento;
- Git e Tags;
- Deploy;
- Atividade prática.

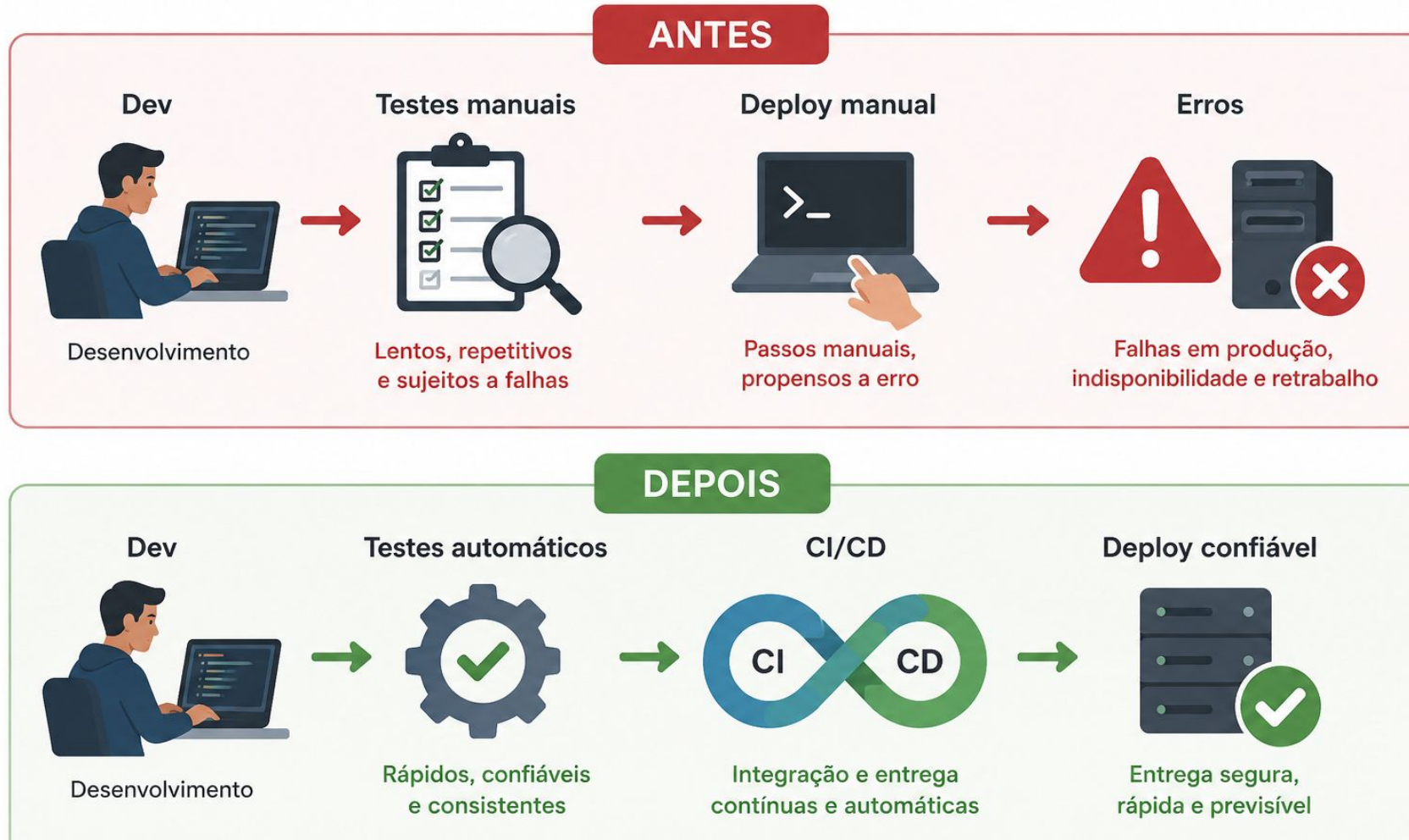
O problema de entregar software

- Entregas demoradas
- Muitos erros em produção
- Falta de comunicação entre equipes
- Processos manuais
- Dificuldade para corrigir bugs rapidamente
- Atualizações geram medo e instabilidade

Antes do DevOps, as áreas de desenvolvimento e infraestrutura trabalhavam separados. Isso causava atrasos, falhas e dificuldade para publicar novas versões do sistema.

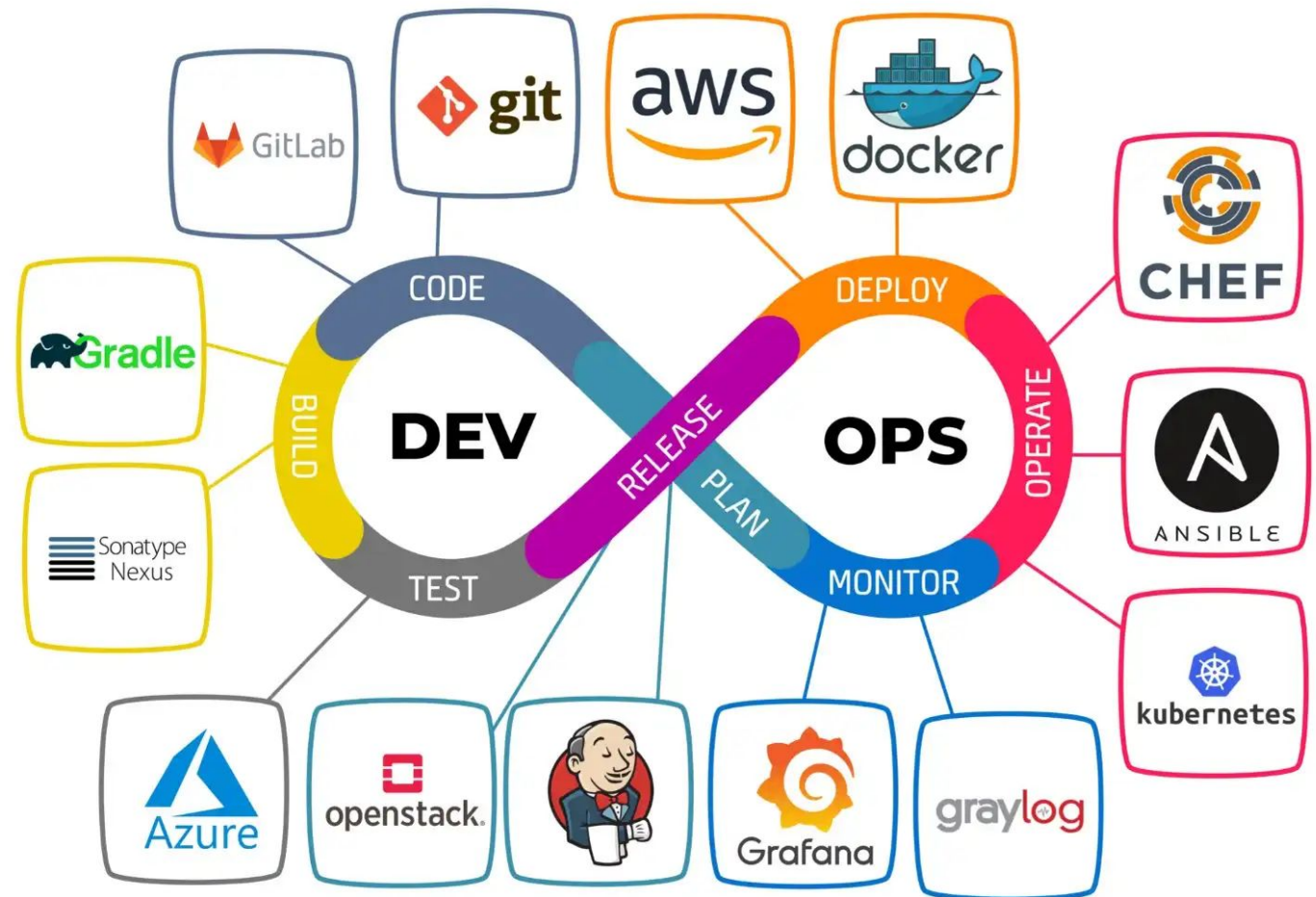
O sistema funcionava no computador do desenvolvedor, mas dá erro no servidor.]

O problema de entregar software



DevOps: O Ciclo Infinito do Desenvolvimento Moderno

DevOps é uma cultura, filosofia e conjunto de práticas que integra as equipes de Desenvolvimento (DEV) e Operações (OPS), eliminando silos organizacionais e acelerando a entrega de software com qualidade e confiabilidade.



PLAN - Planejamento

O planejamento é o ponto de partida do loop infinito. Nesta etapa, as equipes definem o que será desenvolvido, como e quando, utilizando metodologias ágeis para organizar prioridades, distribuir tarefas e alinhar expectativas entre produto, desenvolvimento e operações.

- Jira Gestão de projetos ágeis com sprints, roadmaps e rastreabilidade
- Trello Quadros Kanban visuais para equipes menores e gestão de tarefas
- Azure Boards Planejamento integrado ao ecossistema Microsoft Azure DevOps

CODE - Codificação

- A etapa de codificação é onde as ideias planejadas ganham forma técnica. Nesta fase, os desenvolvedores escrevem, revisam e versionam o código-fonte de forma colaborativa, garantindo rastreabilidade, segurança e qualidade desde o início.
- Boas Práticas de Codificação no DevOps
 - Escrever código limpo e legível
 - Utilizar ferramentas de desenvolvimentos (IDEs)
 - Manter testes
 - Documentar o projeto.

BUILD - Compilação e Build Automatizado

- A etapa de Build transforma o código-fonte em um artefato executável — um arquivo JAR, imagem Docker, pacote NPM ou binário — pronto para ser testado e implantado. A automação deste processo elimina erros humanos e garante builds consistentes e reproduzíveis. Etapas:
 - Compilação: transformação do código-fonte em bytecode ou binário
 - Resolução de dependências: download automático de bibliotecas
 - Empacotamento: geração de JAR, WAR, ZIP, imagem Docker
 - Análise estática: verificação de qualidade de código (SonarQube)
 - Publicação de artefato: versionamento e armazenamento seguro

TEST - Testes Automatizados e Qualidade de Software

- Testes automatizados são o guardião da qualidade no ciclo DevOps. Ao integrar testes diretamente no pipeline de CI/CD, problemas são detectados rapidamente - antes de chegarem ao usuário final. O objetivo é garantir que cada build seja confiável, funcional e seguro.
- **Exemplo:** Uma plataforma de delivery executa +3.000 testes automatizados a cada novo deploy. O pipeline válida: cálculo de frete, processamento de pagamentos, notificações push e fluxo completo de pedido - tudo em menos de 8 minutos, sem intervenção humana.

RELEASE - Liberação e Controle de Versões

- A etapa de Release é o ponto de decisão no pipeline DevOps: o software foi desenvolvido, compilado e testado - agora precisa ser aprovado, versionado e preparado para produção. É o momento em que qualidade e governança garantem que apenas código confiável avança.
- No Release, o artefato gerado no Build recebe um número de versão semântica (ex: v2.4.1) seguindo o padrão Major.Minor.Patch. Gates de qualidade (Quality Gates) verificam cobertura de testes, análise de segurança e aprovação manual de stakeholders antes de avançar.

DEPLOY - Implantação Automatizada

- O Deploy é o momento em que o software aprovado é implantado nos ambientes de produção de forma automatizada, rápida e confiável. Com a cultura DevOps, implantações que antes levavam dias agora acontecem em minutos — com risco reduzido e reversão automática em caso de falha.
- **Exemplo:** Uma empresa de streaming possui 47 microsserviços independentes. Com Kubernetes no AWS EKS, cada serviço é implantado automaticamente via pipeline GitLab: a imagem Docker é construída, publicada no ECR (registry AWS) e o Kubernetes atualiza os pods em rolling update — sem downtime e com rollback automático se o health check falhar.;

OPERATE - Operação e Gestão de Infraestrutura

- Garante que a aplicação em produção permaneça estável, disponível e escalável. Com práticas de automação operacional, as equipes de Ops deixam de apagar incêndios manualmente e passam a gerenciar infraestrutura como código, respondendo proativamente a mudanças de demanda.
- **Exemplo:** Uma plataforma de streaming como o modelo Netflix opera com milhares de servidores globais. Durante horários de pico (novelas, jogos ao vivo), o Kubernetes aumenta automaticamente o número de réplicas dos serviços de transcodificação e entrega de conteúdo. O balanceamento de carga distribui requisições entre instâncias saudáveis, enquanto o Ansible garante configurações uniformes em todos os nós.

MONITOR - Observabilidade e Monitoramento Contínuo

O Monitoramento fecha o loop infinito do DevOps. É a etapa onde a equipe obtém visibilidade total do sistema em produção - coletando métricas, logs e traces para detectar anomalias, entender comportamentos e alimentar o próximo ciclo de planejamento com dados reais.



Métricas

Dados numéricos sobre o estado do sistema: CPU, memória, latência, taxa de erro, requisições/segundo



Logs

Registros textuais de eventos do sistema: erros, transações, autenticações e ações de usuários



Traces

Rastreamento do caminho de uma requisição através de todos os serviços envolvidos no processamento

Integração Contínua (CI)

- Prática de integrar alterações de código frequentemente em um repositório compartilhado.
- Cada novo commit executa automaticamente:
 - Build da aplicação
 - Testes automatizados
 - Validação do código
 - Verificação de falhas

Objetivo: detectar problemas cedo e sempre poder voltar versão

Entrega Contínua (CD)

- Prática de automatizar a preparação e disponibilização do software para produção de forma rápida e confiável.
- Cada nova versão aprovada executa automaticamente:
 - Build da aplicação
 - Testes automatizados
 - Validação do ambiente
 - Geração da release
 - Preparação para deploy

Objetivo: Reduzir riscos na entrega e manter o software sempre pronto para publicação em produção.

Deploy Contínuo

- Prática de publicar automaticamente novas versões em produção após aprovação nos testes e validações do pipeline.
- Cada alteração aprovada executa automaticamente:
 - Build da aplicação
 - Testes automatizados
 - Validação de qualidade
 - Deploy em produção
 - Monitoramento da aplicação

Objetivo: Automatizar o processo de publicação do software em produção.

Introdução ao Versionamento

- O versionamento é o processo de atribuir um identificador único a cada versão de um documento ou software.
 - Pode ser numérico (ex: v1.0, v1.1, v2.0) ou baseado em datas (ex: 2023-10-01).
 - Registra o que foi alterado, por quem e quando.
 - Garante rastreabilidade e auditoria completa das mudanças.
- Facilita a organização, colaboração e recuperação de versões anteriores.

Versionamento Semântico

- Versionamento Semântico (“Semantic Versioning” ou SemVer) é um sistema de controle de versão utilizado para indicar claramente as mudanças em um software.
- As versões seguem o padrão MAJOR.MINOR.PATCH (Ex: 2.1.3), onde:
- MAJOR (Ápice) - Mudanças incompatíveis com versões anteriores.
- MINOR (Incremento) - Novas funcionalidades, mas compatível com versões anteriores.
- PATCH (Correção) - Correção de bugs sem alteração na API.

Versionamento Semântico

Refatoração ou Nova Função: 1.0.0

Melhorias: 0.1.0

Bug Fix: 0.0.1

All releases by project:

App Infr
Infr Staff Panel
MFE-Brands
MFE-Campaign
MFE-Influencers
MFE-Pre-campaigns
MFE-Terms
MFE-Transactions
MS-Auth

[Latest releases:](#)

Staff Panel - Release 3.0.0 - 05/01/2024

- New Feature: Inserir página com histórico geral de todas campanhas e pré-campanhas associadas com o usuário.
- New Feature: Adicionar Recurso de Recuperação de Senha.

MS-Auth - Release 1.0.0 - 05/01/2024

- New Feature: Adicionar Recurso de Recuperação de Senha

MS-Notifications - Release 0.1.0 - 14/12/2023

- New Feature: Implementar Rota de Pub/Sub para Push notificação Mobile.

Fonte: Elaboração própria.

Versionamento Semântico

- Exemplo Prático:
 - 1.0.0 → Primeira versão estável
 - 1.1.0 → Adição de funcionalidade compatível
 - 1.1.1 → Correção de bug
 - 2.0.0 → Mudança incompatível com versões anteriores
- Vantagens do Versionamento Semântico:
 - Facilita a gestão de dependências.
 - Clareza para desenvolvedores e usuários.
 - Ajuda na manutenção e previsibilidade do software.

Exemplos de Alterações no Código

- Bug Fix – Correção de erros no código.
- New Feature – Adição de uma nova funcionalidade ao software.
- Feature Enhancement – Melhorias em funcionalidades existentes.
- Refactoring – Reorganização do código para torná-lo mais limpo e eficiente.
- Performance – Otimização para melhorar velocidade e eficiência.
- Security Patch – Correção de vulnerabilidades de segurança.
- Dependency Update – Atualização de bibliotecas e frameworks utilizados no projeto.
- Adding Tests – Inclusão de testes automatizados para garantir qualidade do código.

Importância do Versionamento

- **Controle de Mudanças**

- Permite que múltiplos colaboradores trabalhem no mesmo projeto.
- Facilita a integração de alterações e evita perda de trabalho.

- **Histórico e Auditabilidade**

- Registra todas as modificações feitas em um arquivo.
- Essencial para auditorias e conformidade regulatória.

- **Recuperação de Dados**

- Permite reverter para versões anteriores em caso de erro.
- Minimiza perda de trabalho e necessidade de retrabalho.

- **Colaboração Eficiente**

- Melhora a comunicação entre membros da equipe.
- Uso de branches para testar mudanças sem afetar a versão principal.

- **Gerenciamento da Qualidade**

- Permite testar versões antes da implementação final.
- Garante que apenas versões validadas sejam disponibilizadas.

- **Aprimoramento Contínuo**

- Possibilita a análise da evolução do projeto ao longo do tempo.
- Identifica padrões de desempenho e oportunidades de melhoria.

- **Integração com Outras Ferramentas**

- Conexão com sistemas de gestão de projetos e integração contínua.
- Centraliza o fluxo de trabalho e mantém controle das mudanças.

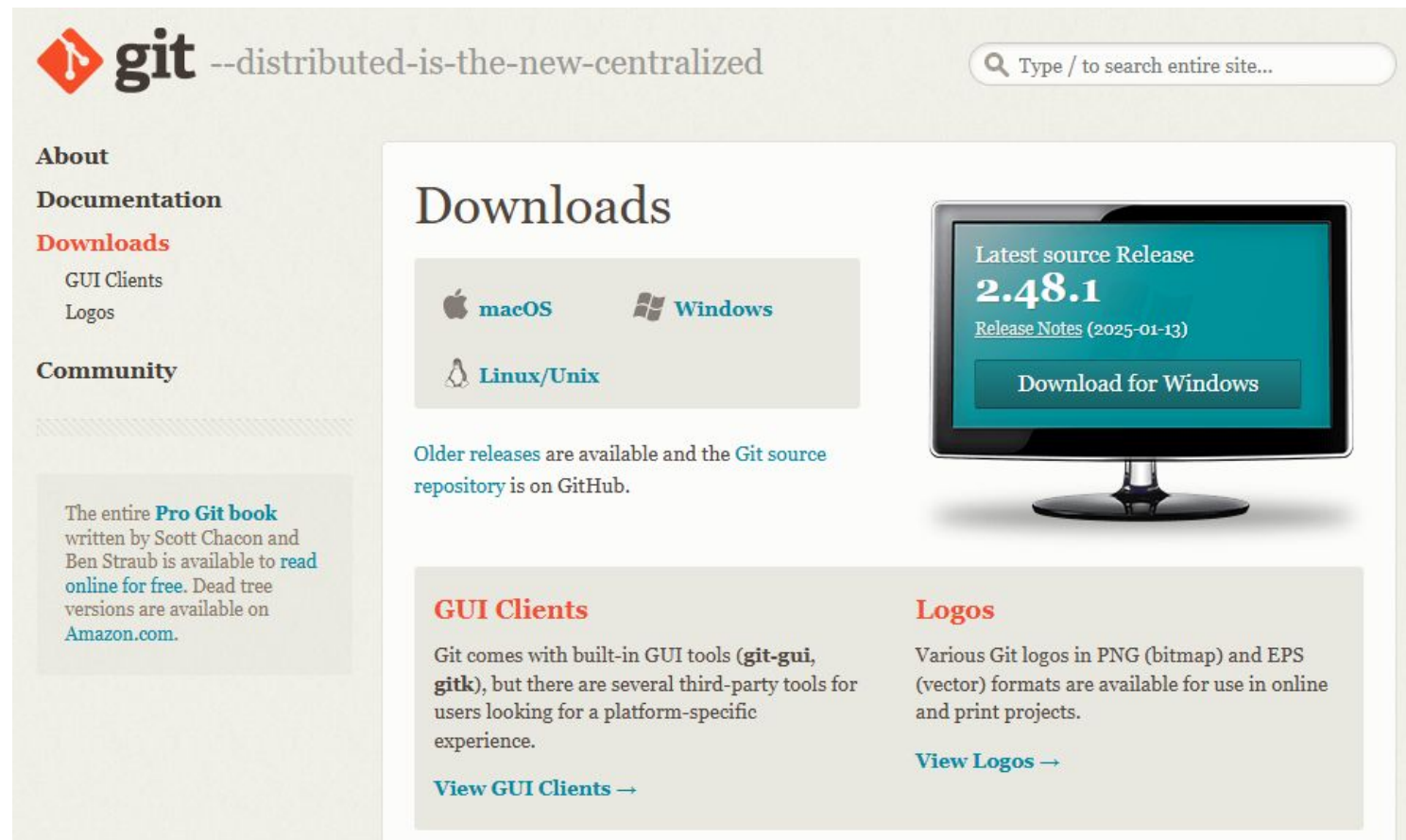
Introdução ao Git

- O que é Git?
 - Sistema de controle de versão de arquivos.
 - Instalado no computador e utilizado via linha de comando.
- O que o Git pode fazer?
 - Sincroniza com repositórios online, permitindo baixar e enviar código.
 - Registra versões do projeto, possibilitando acompanhar mudanças e restaurar versões anteriores.



Fonte: <https://github.com/torvalds>

Instalando o Git



The screenshot shows the Git website's Downloads page. At the top, the Git logo is followed by the tagline "--distributed-is-the-new-centralized". A search bar is in the top right. The left sidebar contains links for "About", "Documentation", "Downloads" (highlighted), "GUI Clients", "Logos", and "Community". Below these is a text box about the "Pro Git book". The main content area has a "Downloads" heading, followed by icons for macOS, Windows, and Linux/Unix. A large monitor graphic displays the "Latest source Release 2.48.1" and a "Download for Windows" button. Below the monitor, text mentions older releases and the GitHub source repository. At the bottom, there are sections for "GUI Clients" and "Logos", each with a "View" link.

git --distributed-is-the-new-centralized

Search: Type / to search entire site...

About
Documentation
Downloads
GUI Clients
Logos
Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Downloads

macOS Windows Linux/Unix

Latest source Release
2.48.1
[Release Notes \(2025-01-13\)](#)
[Download for Windows](#)

Older releases are available and the [Git source repository](#) is on GitHub.

GUI Clients

Git comes with built-in GUI tools (**git-gui**, **gitk**), but there are several third-party tools for users looking for a platform-specific experience.
[View GUI Clients →](#)

Logos

Various Git logos in PNG (bitmap) and EPS (vector) formats are available for use in online and print projects.
[View Logos →](#)

Fonte: <https://git-scm.com/downloads>

Instalando o Git

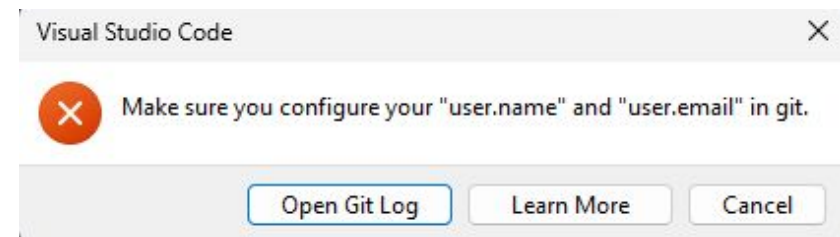
- Escolha a versão do seu sistema operacional
- Baixe e execute o instalador
- Siga as etapas "Next > Next > Install"
- Para testar a instalação:
 - Abra o Prompt de Comando
 - Digite git --version
 - Se aparecer a versão instalada, está correto



Fonte: Elaboração própria.

Instalando o Git

- Se solicitado, configure o usuário e e-mail no CMD com os comandos:
 - `git config --global user.name "<Nome>"`
 - `git config --global user.email "<Email>"`
- Abra o VS Code
- Vá até a aba de Controle de Código-Fonte (terceiro ícone à esquerda)
- Se solicitado, instale o Git
- Feche e reabra o VS Code



Fonte: Elaboração própria.

Criando um Repositório no VS Code

- Crie uma pasta no seu computador (Ex: WebDesign)
- Abra a pasta no VS Code
- Crie os arquivos index.html, style.css e script.js
- Vá até a aba de Controle de Código-Fonte
- Clique em Inicializar Repositório
- Adicione uma mensagem de commit e clique em Confirmar (Commit)

Publicando no GitHub

- Clique em Publicar Branch
- Faça login no GitHub
- Escolha entre repositório público ou privado
- Após a publicação, o código estará acessível no GitHub
- Acesse seu perfil no GitHub
- Vá até "Repositórios"
- Seu projeto deve estar listado
- Clique no repositório e verifique os arquivos

Tags no Git

- O que são tags?
 - Marcadores usados para identificar pontos específicos no histórico do projeto, como versões estáveis (ex: v1.0, v2.0).
- Tipos de tags:
 - Leve (lightweight): Apenas um nome para um commit específico.
 - Anotada (annotated): Armazena informações adicionais, como data, autor e mensagem.
- Uso comum: Marcar releases ou versões importantes do projeto.

Versionamento com Tags no GitHub

- O comando `git tag` lista as tags criadas
- Para criar uma tag, basta utilizar o comando `git tag <nome da tag>`
- Exemplo: `git tag 1.0.0` -> poderia ser utilizada no primeiro commit
- `git push origin <nome da tag>`, adiciona a tag no último commit

Com isso, a cada alteração na aplicação, pode ser detalhado as alterações com maior facilidade, proporcionando uma descrição da alteração e uma sequência de versões desenvolvidas.

O que é Deploy?

- **Definição:** Processo de colocar uma aplicação ou software em produção, tornando-o acessível aos usuários finais.
- **Objetivo:** Garantir que o software funcione corretamente no ambiente de produção.
- **Etapas comuns:**
 - Compilação do código.
 - Configuração do ambiente.
 - Testes finais.
 - Publicação.

Vercel para Deploy e Hospedagem

- Hospedagem Simplificada: Vercel é uma plataforma focada em facilitar o deploy de sites estáticos e aplicações modernas.
- Integração com Git: Conecta-se facilmente com GitHub, GitLab e Bitbucket, realizando deploys automáticos a cada push.
- Suporte a Frameworks Modernos: Ideal para aplicações desenvolvidas com Next.js, Nuxt.js, React, Vue e outros frameworks populares.
- Deploys Instantâneos: Permite rollback e deploys rápidos, garantindo atualizações ágeis e seguras.

Atividade

Nesta atividade, você e seu grupo deverá instalar o Visual Studio Code, criar os arquivos `index.html`, `style.css` e `script.js` copiando os códigos fornecidos para cada um deles e salvando-os na mesma pasta. Em seguida, configure o Git no VS Code para integrar o projeto a um repositório no GitHub, inicializando o versionamento e realizando a publicação do código. Depois, altere o projeto, registre as modificações e salve como uma nova versão utilizando commits e tags. Por fim, documente tudo em um arquivo e realize o envio na plataforma Teams.

Referências

- HUMBLE J; PRIKLANDNICKI R. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013
- MUNIZ, A.; et al. Jornada Devops: Unindo Cultura Ágil, Lean e Tecnologia Para Entrega De Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. DevOps na prática: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. Entrega contínua em Android: Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.
- ARUNDEL, J. DOMINGUS, J. DevOps nativo de nuvem com Kubernetes. São Paulo: Novatec, 2019.

Referências

- MORAES, G. Caixa de Ferramentas DevOps: Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015.
- PIRES, A.; MILITÃO, J. Integração Contínua com Jenkins. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. Descomplicando o Docker. 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. Git: guia prático. São Paulo: Novatec, 2019.
- KIM, G; HUMBLE, J; DEBOIS, P; WILLIS, J; Manual de DEVOPS: Como obter agilidade, confiabilidade e segurança em organizações tecnológicas. São Paulo: Starlin Alta Editora, 2018.

Conceitos de Integração e Entrega Contínua de Software

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br